

ОПЕРАЦИОННЫЕ СИСТЕМЫ

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

ЛАБОРАТОРНАЯ РАБОТА № 4. Процессы

Цель работы:

Получение практических навыков при использовании Win32 API для исследования процессов

Методические указания к выполнению лабораторной работы

Процесс – это исполняемый экземпляр приложения и набор ресурсов, которые выделяются данному исполняемому приложению. Ресурсы включают в себя следующее: 1. виртуальное адресное пространство; 2. системные ресурсы, такие как растровые изображения, файлы, области памяти и т.д.; 3. модули процесса, то есть исполняемые модули, которые отображены (загружены) в его адресное пространство; 4. уникальный идентификационный номер, называемый идентификатором процесса; 5. один или несколько потоков управления. Поток – это внутренняя составляющая процесса, которой операционная система выделяет процессорное время. Каждый процесс должен иметь, по крайней мере, один поток. Поток включает: 1. текущее состояние регистров процессора; 2. два стека, один из которых используется при выполнении в режиме ядра, второй – при выполнении в пользовательском режиме; 3. участок памяти для работы подсистем, библиотеки времени выполнения; 4. динамические библиотеки; 5. уникальный идентификатор, называемый идентификатором потока. Состояние регистров, содержимое стека и области памяти называют контекстом потока (thread's context). Основное назначение потоков – дать процессу возможность поддерживать несколько ветвей управления, то есть выполнять больше задач одновременно. Дескрипторы и идентификаторы процессов Дескриптор процесса определяется функцией CreateProcess, идентификатор текущего процесса – функцией GetCurrentProcessId. Чтобы определить идентификатор другого процесса, необходимо получить список всех процессов и выбрать из него тот процесс, характеристики которого требуются или воспользоваться или воспользоваться методикой получения идентификатора процесса от окна. Между дескриптором и идентификатором процесса (или потока) существуют следующие основные различия: 1. дескриптор действует в пределах своего процесса, в то время как идентификатор работает на системном уровне. Таким образом, дескриптор действует только в пределах того процесса, в котором он был создан; 2. у каждого процесса только один идентификатор, но может быть несколько дескрипторов; 3. некоторым API-функциям требуется идентификатор, в то время как другим – дескриптор процесса. Идентификация процесса Идентификация процесса возможна различными способами с использованием следующих объектов: 1. идентификатор процесса; 2. дескриптор процесса; 3. полное имя загрузочного файла; 4. дескриптор модуля загрузочного файла процесса. Имена файлов и дескрипторы модулей Каждый модуль (DLL, OCX, DRV и т.д.), загруженный в пространство процесса, имеет свой дескриптор (module handle), называемый также логическим номером экземпляра (instance handle). В 16-разрядной Windows данными терминами обозначались разные объекты, в 32-разрядной системе это один и тот же объект. Дескриптором исполняемого модуля является начальный или базовый адрес данного исполняемого модуля в адресном пространстве процесса. Это объясняет, почему такой дескриптор имеет смысл только в рамках процесса, включающего данный модуль. Перейти от имени файла модуля к дескриптору модуля и наоборот не составляет особого труда, по крайней мере, в пределах одного процесса. Функция GetModuleFileName принимает дескриптор модуля, чтобы вернуть полное имя (имя и путь) исполняемого файла.

Псевдодескрипторы процессов Функция `GetCurrentProcess` возвращает псевдодескриптор текущего процесса. Псевдодескриптор (`pseudohandle`) представляет собой упрощенный вариант дескриптора. По определению, псевдодескриптор – это зависимое от процесса число, которое служит идентификатором процесса и может использоваться в вызовах тех API-функций, которым требуется дескриптор процесса. Хотя назначение псевдодескрипторов и обычных дескрипторов почти одно и то же, у них все же есть некоторые существенные различия. Псевдодескрипторы не могут наследоваться порожденными процессами, как настоящие дескрипторы (`real handler`). К тому же псевдодескрипторы ссылаются только на текущий процесс, а настоящие дескрипторы могут ссылаться и на внешний (`foreign`). Windows предоставляет возможность получения настоящего дескриптора по псевдодескриптору при помощи API-функции `DuplicateHandle`.

Перечисление процессов в Windows 9x. Моментальные снимки Благодаря многозадачной природе среды Win32 такие объекты, как процессы, потоки, модули и т.п., постоянно создаются, разрушаются и модифицируются. И поскольку состояние компьютера непрерывно изменяется, системная информация, которая, возможно, будет иметь значение в данный момент, через секунду уже никого не заинтересует. Например, предположим, что вы хотите написать программу для регистрации всех модулей, загруженных в систему. Поскольку операционная система в любое время может прервать выполнение потока, обрабатывающего программу, чтобы предоставить какие-то кванты времени другому потоку в системе, модули теоретически могут создаваться и разрушаться даже в момент выборки информации о них. В этой динамической среде имело бы смысл на мгновение заморозить систему, чтобы получить такую системную информацию. В `ToolHe1p32` не предусмотрено средств замораживания системы, но есть функция, с помощью которой можно сделать "снимок" системы в заданный момент времени – `CreateToolhelp32Snapshot()`.

Ход выполнения лабораторной работы

КОД ПРОГРАММЫ:

```
#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
#include <tchar.h>
#include <vector>

// Функция для вывода сообщения об ошибке
void PrintErrorMessage(const TCHAR* msg) {
    DWORD errorMessageID = GetLastError();
    if (errorMessageID == 0)
        return; // Нет ошибки, поэтому нет сообщения
    LPSTR messageBuffer = nullptr;
    size_t size = FormatMessageA(FORMAT_MESSAGE_ALLOCATE_BUFFER |
    FORMAT_MESSAGE_FROM_SYSTEM | FORMAT_MESSAGE_IGNORE_INSERTS,
    NULL, errorMessageID, MAKELANGID(LANG_NEUTRAL,
    SUBLANG_DEFAULT), (LPSTR)&messageBuffer, 0, NULL);
    std::cerr << msg << ": " << messageBuffer << std::endl;
    LocalFree(messageBuffer);
}

// Функция для вывода информации о модулях процесса
void ListModules(DWORD processID) {
    HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE, processID);
```

```

if (snapshot == INVALID_HANDLE_VALUE) {
    PrintErrorMessage(TEXT("CreateToolhelp32Snapshot failed for modules"));
    return;
}

MODULEENTRY32 moduleEntry;
moduleEntry.dwSize = sizeof(MODULEENTRY32);

if (!Module32First(snapshot, &moduleEntry)) {
    PrintErrorMessage(TEXT("Module32First failed"));
    CloseHandle(snapshot);
    return;
}

do {
    _tprintf(TEXT("\n%s (0x%08X)", moduleEntry.szModule,
(DWORD)moduleEntry.modBaseAddr);
} while (Module32Next(snapshot, &moduleEntry));

CloseHandle(snapshot);
}

int main() {
    // Шаг 1: Получение идентификатора текущего процесса
    DWORD currentProcessID = GetCurrentProcessId();
    _tprintf(TEXT("Current Process ID: %u\n"), currentProcessID);

    // Шаг 2: Получение псевдодескриптора текущего процесса
    HANDLE pseudoHandle = GetCurrentProcess();
    if (pseudoHandle == NULL) {
        PrintErrorMessage(TEXT("GetCurrentProcess failed"));
        return 1;
    }

    // Шаг 3: Получение дескриптора текущего процесса с помощью DuplicateHandle
    HANDLE currentProcessHandle;
    if (!DuplicateHandle(pseudoHandle, pseudoHandle, pseudoHandle,
&currentProcessHandle, 0, FALSE, DUPLICATE_SAME_ACCESS)) {
        PrintErrorMessage(TEXT("DuplicateHandle failed"));
        CloseHandle(pseudoHandle);
        return 1;
    }

    // Шаг 4: Получение копии дескриптора текущего процесса с помощью OpenProcess
    HANDLE openProcessHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
currentProcessID);
    if (openProcessHandle == NULL) {
        PrintErrorMessage(TEXT("OpenProcess failed"));
        CloseHandle(currentProcessHandle);
        CloseHandle(pseudoHandle);
        return 1;
    }
}

```

```
}

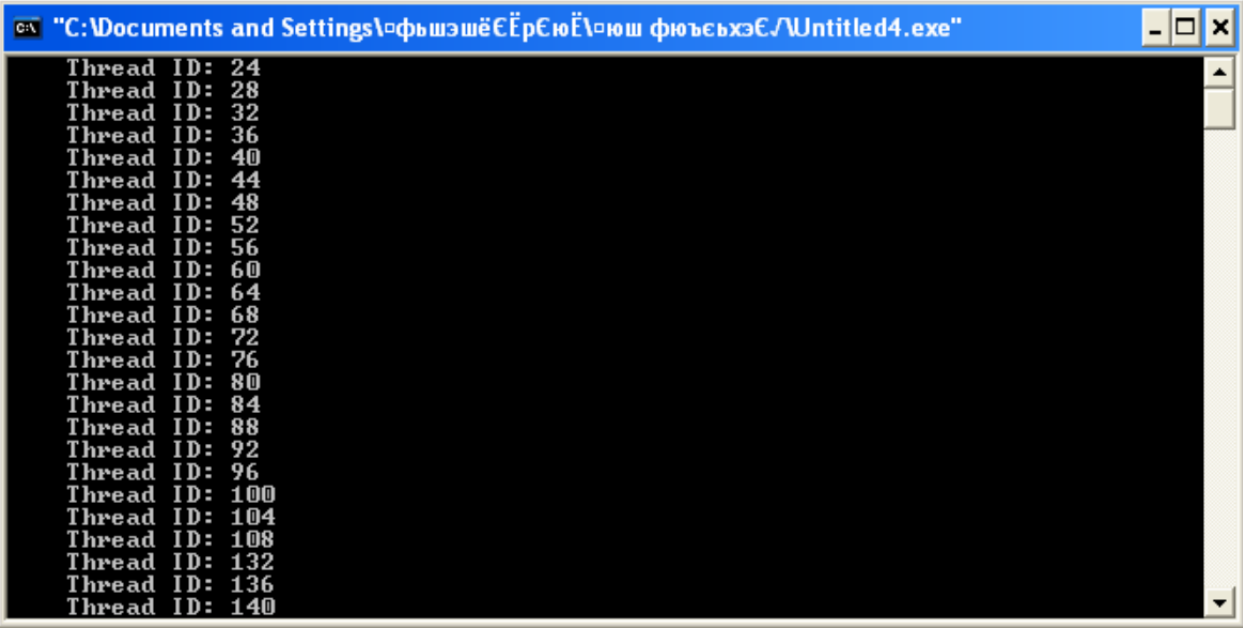
// Шаг 5: Закрытие дескриптора, полученного функцией DuplicateHandle
CloseHandle(currentProcessHandle);

// Шаг 6: Закрытие дескриптора, полученного функцией OpenProcess
CloseHandle(openProcessHandle);

// Шаг 7: Вывод информации о модулях текущего процесса
ListModules(currentProcessID);

return 0;
}
```

РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ:



```
Thread ID: 24
Thread ID: 28
Thread ID: 32
Thread ID: 36
Thread ID: 40
Thread ID: 44
Thread ID: 48
Thread ID: 52
Thread ID: 56
Thread ID: 60
Thread ID: 64
Thread ID: 68
Thread ID: 72
Thread ID: 76
Thread ID: 80
Thread ID: 84
Thread ID: 88
Thread ID: 92
Thread ID: 96
Thread ID: 100
Thread ID: 104
Thread ID: 108
Thread ID: 132
Thread ID: 136
Thread ID: 140
```